

Démarche du projet — SAE4D01 DevCloud

Journal de bord : comment on est arrivé au résultat, étape par étape

Ce document raconte la **progression réelle** du projet : chaque décision, le **pourquoi**, le **comment**, et le **résultat**. Il complète le compte-rendu technique (CR-COMPLET).

Phase 0 — Point de départ

Au départ : un lab **containerlab** sur la VM220, avec **5 topologies** déjà construites en **FRR** (eBGP, iBGP-RR, OSPF, mixed, EVPN), toutes sur la même architecture **leaf-spine** (2 spines, 3 leafs, 3 hosts). Objectif initial : valider que chaque protocole route correctement (ping de bout en bout).

Constat : ça marche, mais c'est *qualitatif*. On ne sait pas **laquelle est la meilleure**, ni **comment le prouver**. D'où la suite.

Phase 1 — Mesurer plutôt que supposer (benchmark)

Décision : ajouter une **mesure chiffrée** du débit de chaque techno. **Comment** : un script `benchmark.sh` qui, pour chaque topo, **déploie** □ **attend 35 s de convergence** □ **lance iperf3 (TCP 30 s + UDP 30 s)** □ **sauvegarde le résultat en JSON** □ **détruit**. Séquentiel pour ne pas fausser les mesures (une seule topo active à la fois). **Résultat** : un premier tableau de débits. Tous proches de 10 Gbit/s □ première intuition : *le protocole de routage n'est peut-être pas le facteur déterminant du débit*.

Phase 2 — Se situer face à l'existant

Décision : comparer notre approche à celle du binôme (Valentin) pour identifier ce qui nous distingue. **Constat** : lui utilise netlab + Arista cEOS, fait surtout de l'EVPN, et a un **monitoring (Grafana)** mais **aucune mesure de performance chiffrée**. Nous : 5 topos FRR + un benchmark, mais **pas de supervision visuelle**. **Décision qui en découle** : ajouter une **observabilité temps réel** ET la **relier au benchmark** — c'est le différenciateur (mesurer et visualiser, là où lui ne fait que visualiser).

Phase 3 — Mettre en place la supervision (Prometheus + Grafana)

Comment : déploiement d'une stack en docker compose — **Prometheus** (collecte/stockage) + **Grafana** (affichage) + **node-exporter** (métriques hôte). **Premier obstacle** : pour avoir le débit **par conteneur**, on tente **cAdvisor** (l'outil standard). **Échec** : cAdvisor bute sur un bug `overlay2/cgroup v2 (failed to identify the read-write layer ID)`, il ne reconnaît pas les conteneurs containerlab □ **aucune métrique réseau par nœud**.

Décision : plutôt que renoncer, **écrire notre propre exporter**. `clab_exporter.py` (Python) : - liste les conteneurs `clab-*` ; - pour chacun, entre dans sa **network-namespace** (`nsenter`) et lit `/proc/net/dev` (compteurs kernel RX/TX par interface) ; - expose ces données au **format Prometheus** sur le port :9101. **Résultat** : Prometheus scrape l'exporter toutes les 2 s, Grafana affiche le débit live par nœud. **Chaîne complète** : conteneurs → `clab_exporter` → Prometheus → Grafana.

Phase 4 — La découverte ECMP (le moment clé)

Test : on lance un `iperf3` et on regarde, dans Grafana, la répartition du trafic sur les 2 spines. **Surprise** : **100 % du trafic passe par un seul spine**, même en lançant **8 flux parallèles**. **Diagnostic, étape par étape** : 1.

show ip route montre bien **2 chemins ECMP** (via spine1 et spine2) dans la table `□` la route est OK. 2. Donc le problème n'est pas le routage, mais la **répartition** du kernel. 3. Le kernel Linux hashé par défaut en **L3** (IP source + destination uniquement). Comme tous les flux ont la **même paire host** `□ host`, ils tombent tous sur le **même** spine. **Correction** : activer le hash **L4** (`sysctl net.ipv4.fib_multipath_hash_policy=1`), qui inclut les **ports** `□` des flux à ports différents se répartissent. **Résultat mesuré** : on passe de **100/0** à **62/38** entre les deux spines. ECMP prouvé, visible en direct dans Grafana.

Phase 5 — Benchmark des 5 topos et 2e découverte (MTU VXLAN)

Décision : étendre le benchmark à **toutes** les topos, EVPN compris. **Problème** : sur EVPN, le **TCP sort à 0 Gbit/s**, alors que le **ping passe** et que l'**UDP marche**. **Diagnostic** : 1. Ping OK + UDP OK `□` la connectivité L2/VXLAN fonctionne. 2. Seul le TCP à **plein débit** échoue `□` c'est un problème de **taille de paquet**. 3. L'interface `vxlan100` était en **MTU 1500**, mais VXLAN ajoute ~50 octets d'encapsulation. Les segments TCP pleins dépassent la MTU du tunnel et sont **silencieusement jetés** ; le mécanisme de découverte de MTU (PMTUD) ne récupère pas `□` **blackhole TCP**. **Correction** : monter la MTU de `vxlan100` à 9000 `□` **TCP passe de 0 à 9.76 Gbit/s**. **Leçon** : « ça ping donc ça marche » est un piège ; ce bug ne se voit qu'en **mesurant le débit**. **Décision méthodologique** : finalement **exclure EVPN du tableau comparatif** — c'est un overlay L2, pas comparable à un routage L3 (surcoût d'encapsulation). On garde **4 technos de routage** comparables.

Phase 6 — Rendre la comparaison lisible (dashboard)

Besoin : ne pas juste voir le trafic live, mais **comparer les technos d'un coup d'œil**. **Comment** : on **étend l'exporter** pour qu'il lise aussi les **JSON du benchmark** et expose des métriques `clab_bench_tcp_gbps{topo=...}`. Grafana peut alors dessiner des **barres comparatives**. **Itérations** (demandes successives) : d'abord 2 dashboards (live + comparaison), puis **fusion en un seul dashboard** à 4 sections (comparaison, trafic live, preuve ECMP, ressources) pour simplifier.

Phase 7 — Séparer le lab de la production (VM dédiée)

Décision : la VM220 devant servir à de la **production**, on **migre tout le lab** sur une VM **dédiée** (mauvaise pratique de faire cohabiter benchmarks saturant le CPU et prod). **Obstacle** : la VM221 existait mais était une **coquille issue d'un clone planté** (disque partiel, verrou `clone`, IP en conflit avec la 220). **Comment, proprement** : 1. déverrouillage, nettoyage du disque cassé ; 2. provisionnement **frais** depuis une image **Debian 12 cloud** (cloud-init), correction de l'IP en `.221` ; 3. réinstallation de **Docker + containerlab + FRR** ; 4. copie des **4 topos de routage** (EVPN exclu) + de la stack monitoring ; 5. relance d'un **benchmark propre**. **Résultat** : lab isolé sur VM221, **VM220 libérée pour la prod**. Benchmark cohérent : **~11 Gbit/s pour les 4 technos** (écarts < 0,1 G), ce qui **confirme** l'hypothèse de la Phase 1.

Phase 8 — Documenter et présenter

- **Schémas** : générés depuis `containerlab graph` puis stylés (zones SPINE/LEAF/HOSTS, loopbacks, AS, tag du protocole sur les liens, distinction **underlay/overlay**).
 - **Accès propre** : alias SSH `vm220/vm221` (corrige une erreur de connexion VS Code), édition via **Remote-SSH**.
 - **Compte-rendu** : structuré **par lab** (objectif `□` plan `□` config `□` vérification `□` conclusion), enrichi de ce que le binôme n'a pas (benchmark chiffré, observabilité, bugs diagnostiqués).
-

Conclusion de la démarche

La progression suit une logique simple : **construire** □ **douter** □ **mesurer** □ **diagnostiquer** □ **corriger** □ **isoler** □ **documenter**. Les deux temps forts ne sont pas des configurations, mais des **bugs trouvés en mesurant** (hash ECMP L3, MTU VXLAN) — ce qui n'apparaît jamais avec un simple ping. C'est la **mesure** qui a fait la différence, et c'est elle qui distingue ce travail.